

Type-Safe Arithmetic Expression Interpreter

EC 8206 — Functional Programming
Project Assignment (20%)

Student name [Your Name]

Registration number [Your Registration Number]

Language Haskell (GHC)

Design summary

The project models expressions with the algebraic data type `Expr`. Its alternatives are numeric literals, variables, the four arithmetic operators, and a local `Let` binding. An environment is a list of name/value pairs. Placing a new pair at the front implements lexical shadowing without changing an existing environment.

Safety: `eval :: Env -> Expr -> Either String Double` returns either a useful error message or a number. Division by zero and undefined variables therefore remain ordinary, typed results instead of terminating the program.

How to run

Put `Expr.hs` and `Main.hs` in the same directory. With GHC installed, execute:

```
runghc Main.hs

# or compile with warnings enabled
ghc -Wall Main.hs -o expression-interpreter
./expression-interpreter
```

Implementation Evidence

Sample evaluations: expected versus actual

Environment for these cases: `[("x", 10), ("y", 4)]`.

#	Expression	Expected	Actual
1	<code>Add (Lit 2) (Lit 3)</code>	Right 5.0	Right 5.0
2	<code>Mul (Var "x") (Var "y")</code>	Right 40.0	Right 40.0
3	<code>Sub (Mul (Var "x") (Lit 3)) (Var "y")</code>	Right 26.0	Right 26.0
4	<code>Div (Lit 20) (Var "y")</code>	Right 5.0	Right 5.0
5	<code>Let "x" (Lit 7) (Add (Var "x") (Lit 1))</code>	Right 8.0	Right 8.0
6	<code>Div (Lit 8) (Sub (Var "y") (Lit 4))</code>	Left "division by zero"	Left "division by zero"
7	<code>Add (Var "missing") (Lit 1)</code>	Left "undefined variable: missing"	Left "undefined variable: missing"

The simplifier changes `Mul (Add (Var "x") (Lit 0)) (Lit 1)` to `Var "x"`. Across the seven expressions above, `evalBatch` returns `[5.0, 40.0, 26.0, 5.0, 8.0]` and `batchSummary` returns `BatchSummary {succeeded = 5, failed = 2}`.

Part D — Comparative Analysis

Equivalent imperative evaluator (Java-like pseudocode)

```
double eval(Map<String, Double> env, Node expr) {
    switch (expr.kind) {
        case LIT: return expr.value;
        case VAR:
            if (!env.containsKey(expr.name))
                throw new Error("undefined variable: " + expr.name);
            return env.get(expr.name);
        case ADD: return eval(env, expr.left) + eval(env, expr.right);
        case SUB: return eval(env, expr.left) - eval(env, expr.right);
        case MUL: return eval(env, expr.left) * eval(env, expr.right);
        case DIV:
            double denominator = eval(env, expr.right);
            if (denominator == 0) throw new Error("division by zero");
            return eval(env, expr.left) / denominator;
        default: throw new Error("unknown expression tag");
    }
}
```

Comparison

Error representation and propagation. The Haskell signature announces that evaluation can fail: callers must handle both `Left String` and `Right Double`. The `do` expressions automatically stop at the first `Left` and pass that error outward. The Python version throws exceptions, which are invisible in its function signature; a caller can forget to catch them. It must also reject unknown tags at runtime.

State and control flow. Haskell follows the tree through recursive pattern matching. Each call returns a new value, and extending an environment creates a new list whose tail is the previous environment. The imperative version uses a `switch` and a mutable local variable for the denominator. Its control flow and manually tagged `Node` representation require runtime checks.

Compile-time guarantees. Haskell checks that every successful branch returns a `Double`, every failure is a `String`, and the caller cannot accidentally use an `Either String Double` as a number. With warnings enabled, GHC also reports incomplete pattern matches when a new `Expr` constructor is not handled. The Python encoding permits malformed tuples, misspelled tags, missing fields, and nonnumeric payloads until the affected line executes.

Trade-off. The imperative representation may look familiar and makes control flow explicit. Haskell more closely matches the expression grammar, while its types move several defect categories from runtime to compile time.

Part E — Reflection

Pure functions

A pure function produces the same result whenever it receives the same inputs and has no observable side effects. Both `eval` and `simplify` are pure. For example, evaluating `Div (Lit 8) (Lit 0)` in the same environment always returns `Left "division by zero"`; it does not print, throw an exception, read a global variable, or modify the environment. This makes individual cases easy to reason about and test as input/output equations.

`simplify` is also pure even though it transforms a tree. It recursively constructs a new expression. Rules such as $x + 0 \rightarrow x$, $x * 1 \rightarrow x$, $x * 0 \rightarrow 0$, $x - 0 \rightarrow x$, and $x / 1 \rightarrow x$ do not alter the original value. Constant subexpressions are folded, except division by zero, which is preserved so that `eval` can still report the error.

Immutability and scope

Values are immutable: once an `Expr` or environment list has been created, evaluation cannot update it in place. For `Let "x" bound body`, the bound expression is evaluated first and the body receives `("x", value) : env`. The old environment remains available unchanged, while lookup selects the first matching pair. This gives predictable local shadowing without mutable variables.

Why the shape of `Expr` matters

`Expr` is a sum type: a value is exactly one constructor at a time. Each constructor carries only the data it requires. `Lit` always contains a `Double`; `Var` always contains a name; binary operators always contain two complete expressions; and `Let` always contains a name, a bound expression, and a body.

Consequently, states such as an addition with one missing operand or a literal containing text cannot be constructed through the type.

Pattern matching then exposes those shapes safely. The evaluator handles all seven constructors, and a compiler warning highlights omissions after the type is extended. Not every semantic error can be removed by the data type: whether a variable exists depends on the supplied environment, and a denominator can evaluate to zero. Those cases are represented explicitly by `Either`, keeping failure inside the function's type.

Higher-order functions and currying

`evalBatch` uses `map (eval env)`. Currying fixes the shared environment first, producing a function `Expr -> Either String Double` that `map` applies to each item. A `foldr` retains successful values, while `batchSummary` folds the same outcomes into success and failure counts. These functions separate traversal from the operation applied to each element.

Assistance disclosure: OpenAI Codex was used on 25 September 2026 to help draft and review the code and report. The student should verify this disclosure against the University's academic-integrity rules and must be able to explain the submitted work. Recommended background text: Graham Hutton, *Programming in Haskell*, 2nd ed., Cambridge University Press, 2016.